Switching an Oracle Integration REST Connection from Basic Authentication to OAuth Client Credentials

Author: Moon Ray Lo, Oracle ACE Apprentice

Date: October 30, 2026

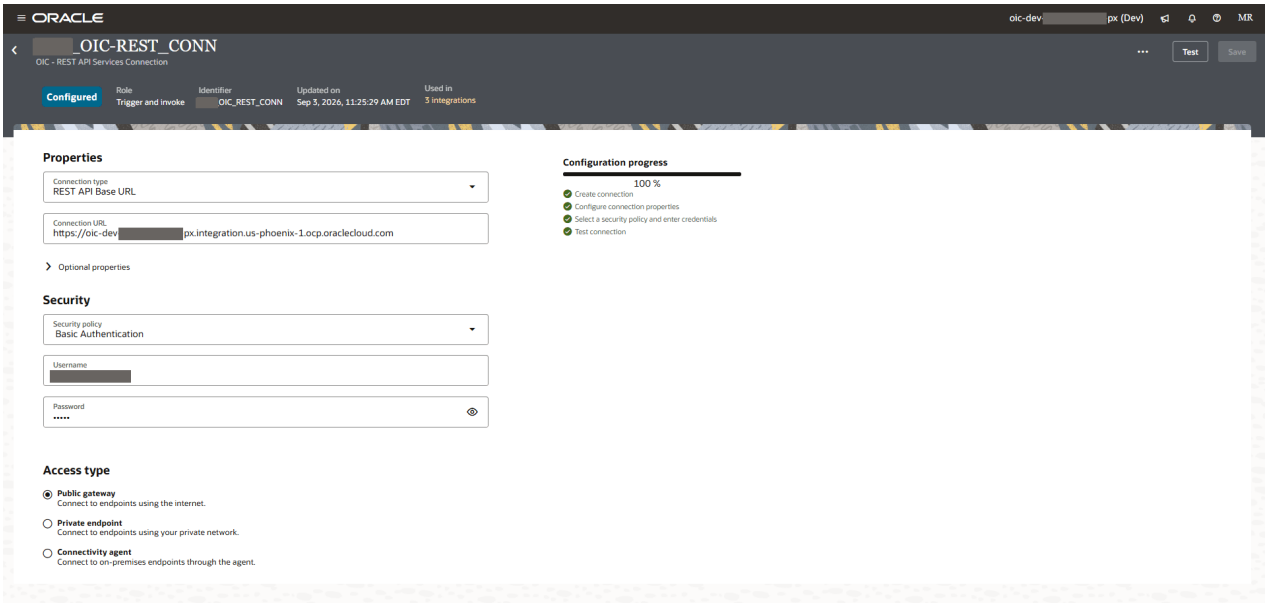
Product: Oracle Integration 3, OCI IAM Identity Domains, OCI Vault (Secrets Management), Postman

Note: environment identifiers (OCIDs, client IDs, identity domain, instance and scope identifiers, and email addresses) are masked in the screenshots. Secret and token values were never shown.

Oracle Integration's REST adapter connection that calls the same instance's own REST endpoints was still on Basic Authentication with a named user account. That works until the password changes, MFA applies or the person leaves, and it runs integrations with that person's access. I moved it to OAuth 2.0 Client Credentials using a dedicated internal confidential application, stored the client secret in OCI Vault, proved the tokens and calls in Postman, and then switched the connection.

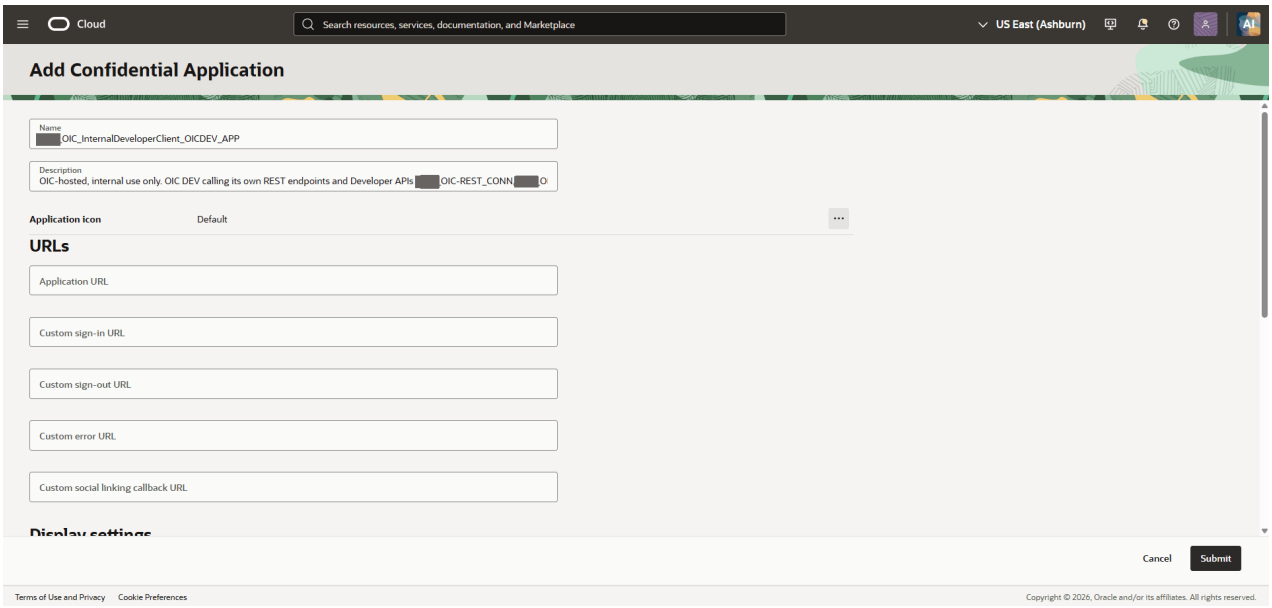
Part 1: Review the connection before the change

I opened the OIC-REST_CONN connection first to record what it calls and how. The Connection URL pointed at the instance's runtime host, so the three integrations using it call integration endpoints, and the security policy was Basic Authentication with a named user. The connection role is Trigger and Invoke, but the security policy only governs the outbound (invoke) calls.

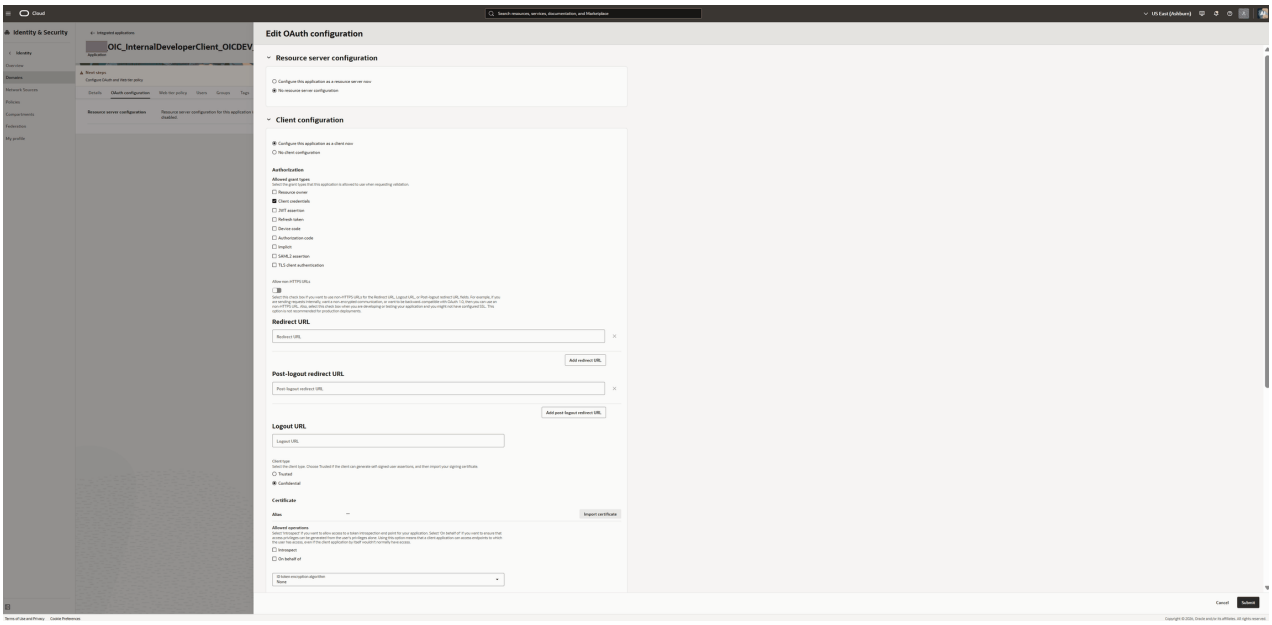


Part 2: Create an internal confidential application and assign its role

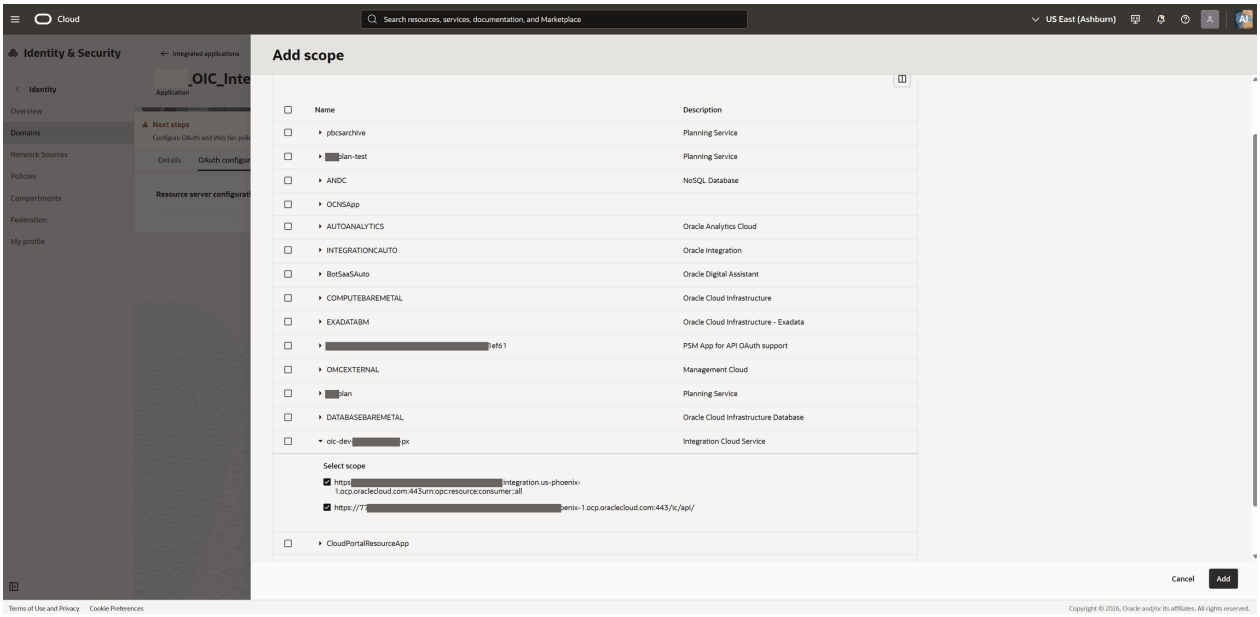
In the identity domain that lists the OIC instance under Oracle cloud services, I created a confidential application named OIC_InternalDeveloperClient_OICDEV_APP. The description states its purpose, callers, grant, role and owner, so anyone reading it later knows it is for internal OIC use only.



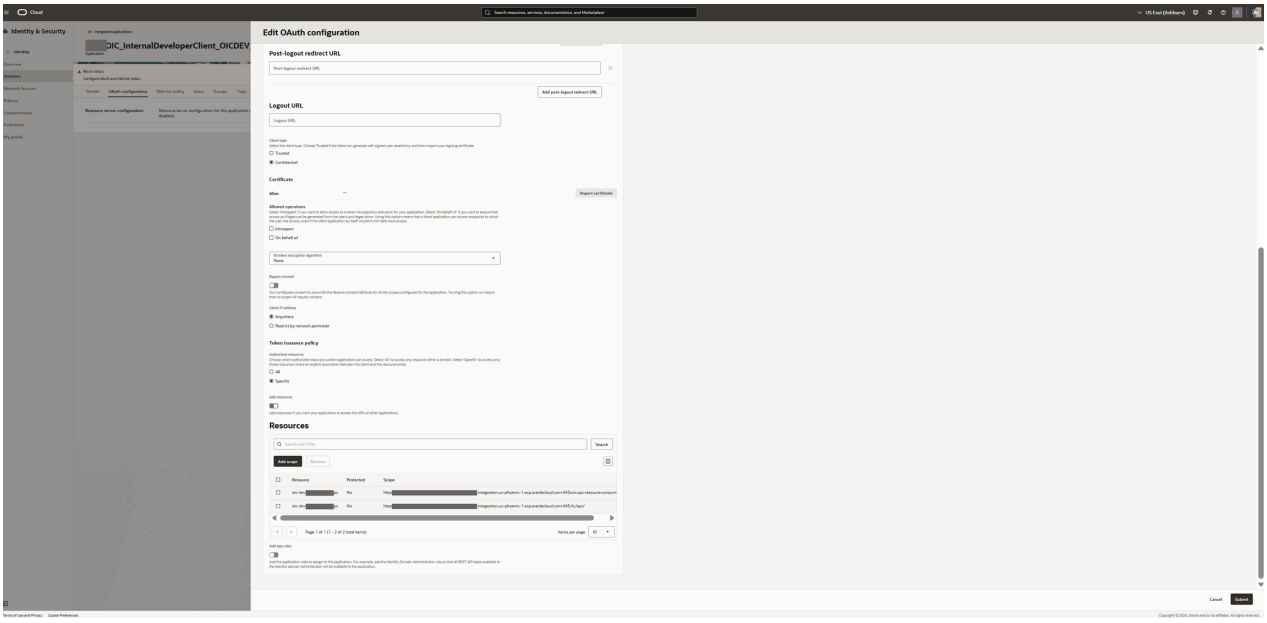
Under Client configuration, I selected Client credentials as the only grant type. The client type stays Confidential.



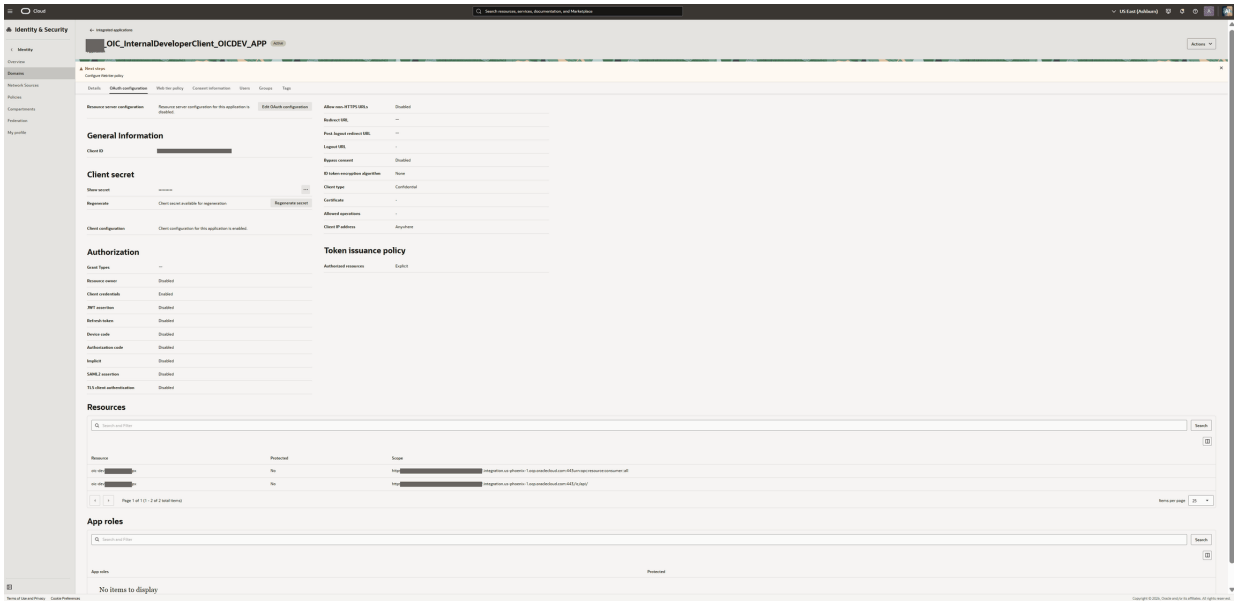
I added both OIC scopes from the instance's resource: consumer::all for running integrations and /ic/api/ for the Developer APIs. A connection uses one scope, so a later Developer API connection can reuse the same application.



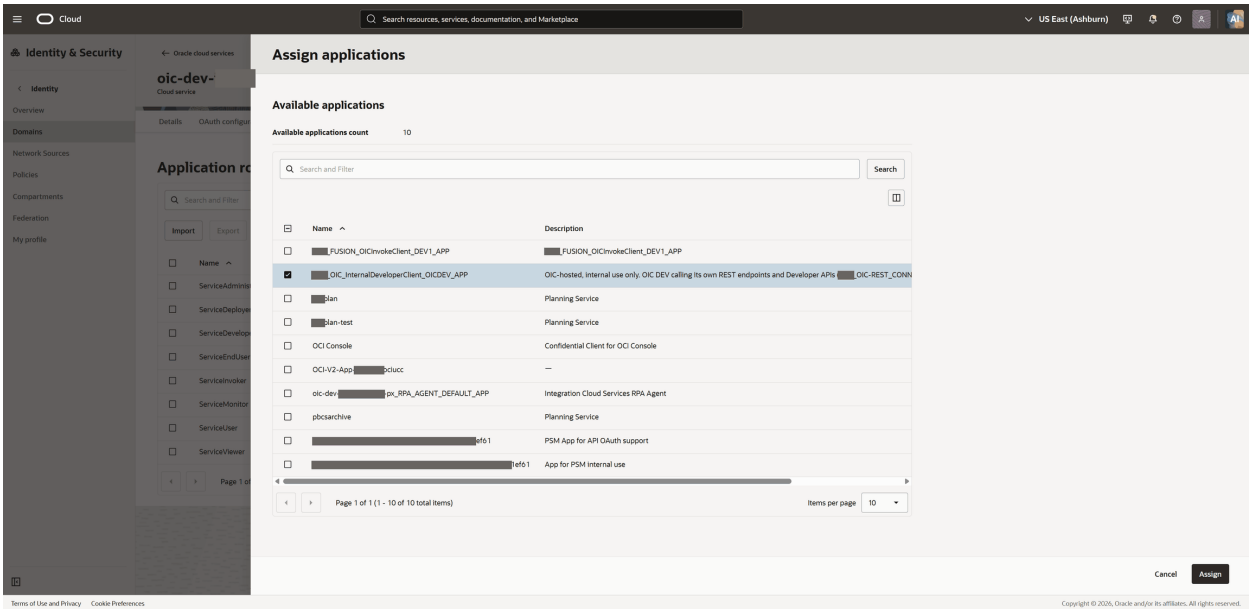
The token issuance policy then listed both scopes as authorized resources.



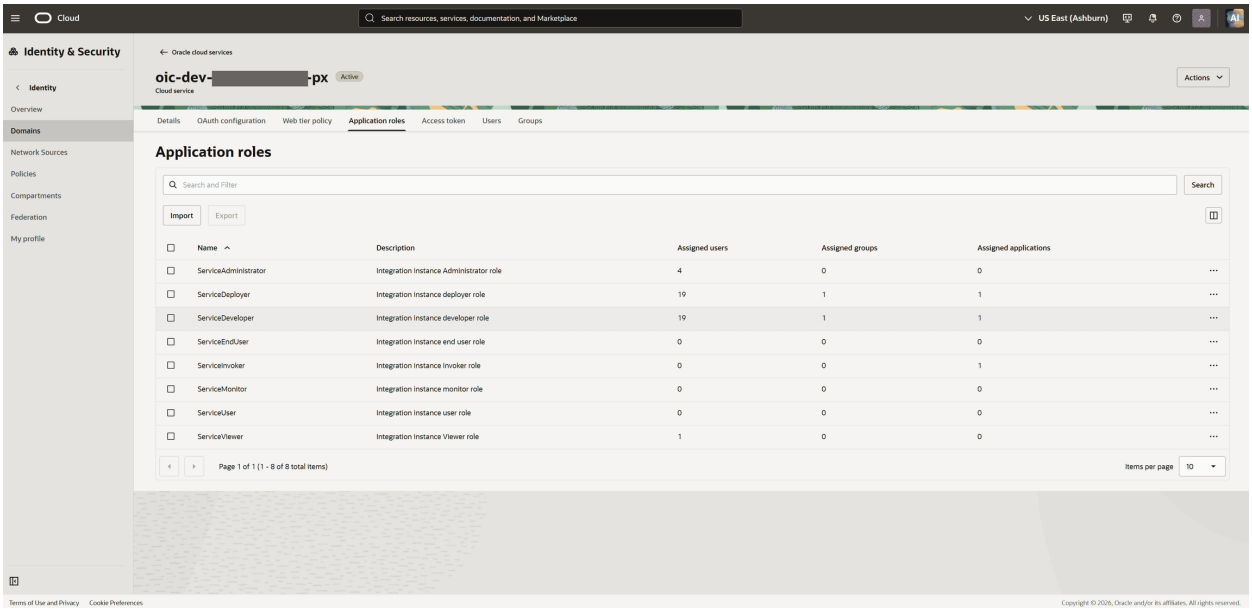
After I activated the application, it showed Client credentials enabled, both scopes, and its client ID.



Scopes and roles are separate checks. On the OIC instance under Oracle cloud services, I opened Application roles and assigned the application to ServiceDeveloper, the lowest role that covers both running integrations and Developer API reads and writes.

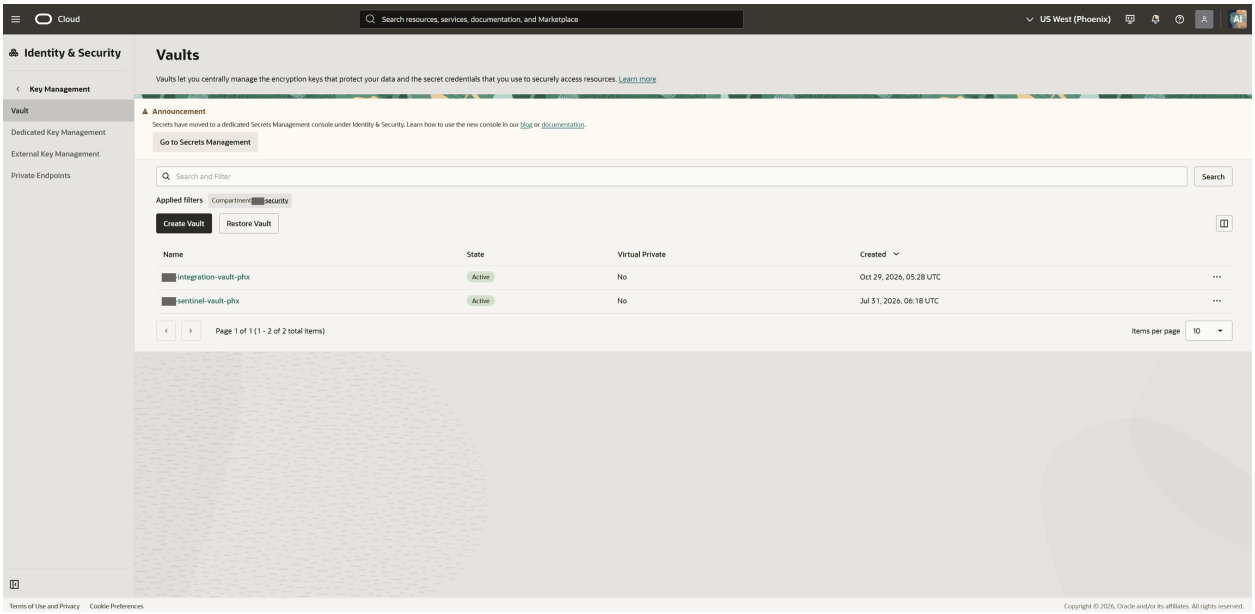


The application roles list confirmed the assignment.

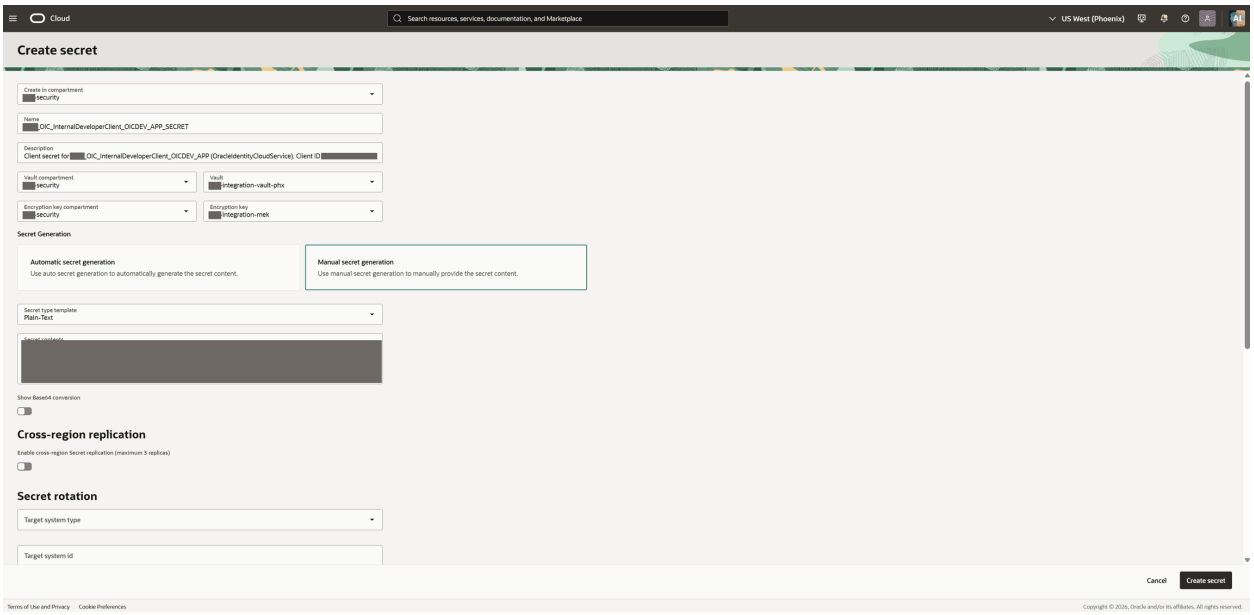


Part 3: Store the client secret in OCI Vault

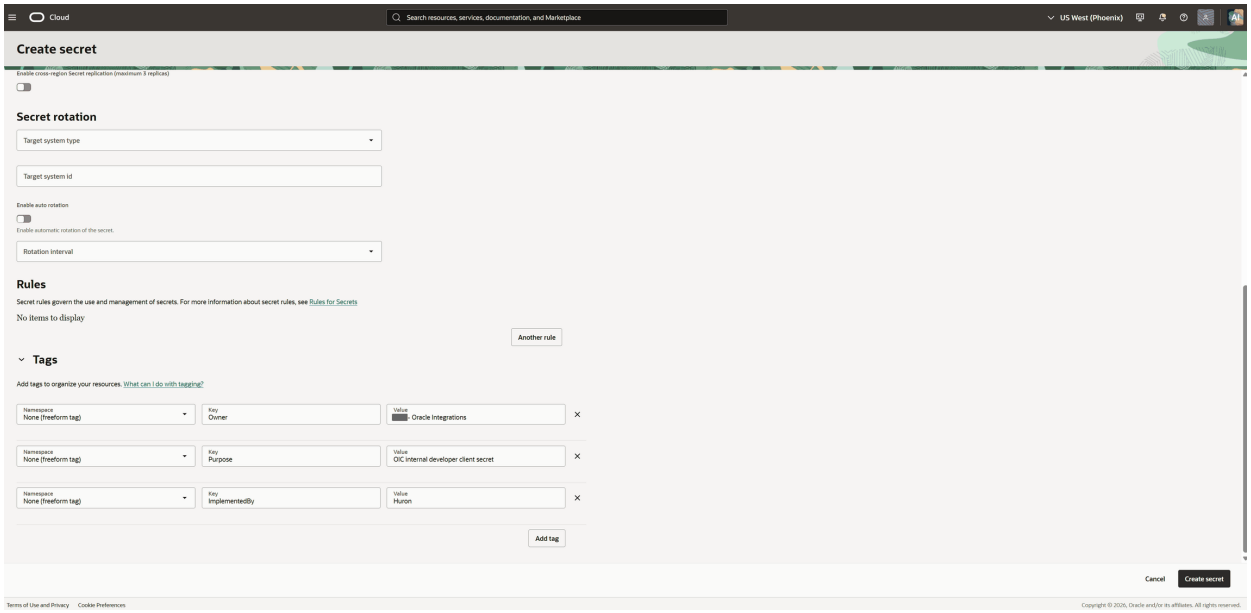
I kept the client secret in OCI Vault rather than in a document or a password manager entry. Two console details are worth knowing: the vault list only shows vaults in the selected compartment, and secrets now have their own Secrets Management console under Identity & Security.



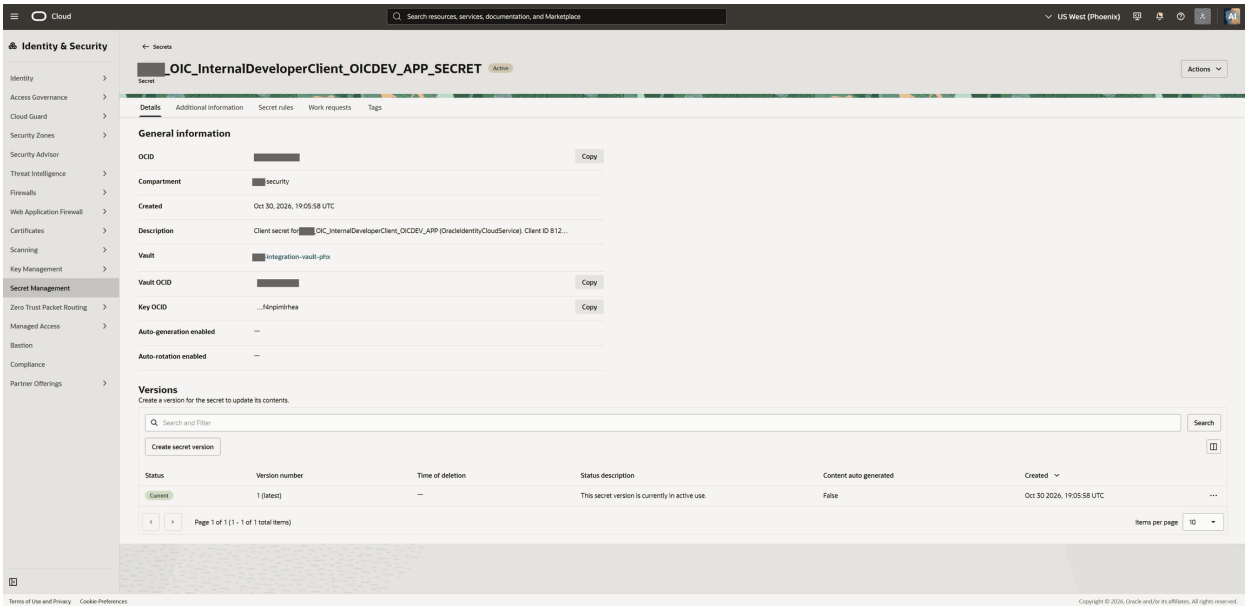
I created the secret with Manual secret generation and a Plain-Text template, then pasted the client secret. The form defaults to Automatic, which would have stored a random passphrase instead.



I tagged the secret with its owner and purpose, and left rotation manual: regenerate the client secret, then create a new secret version.



The secret was Active at version 1.



Part 4: Prove the tokens and calls in Postman

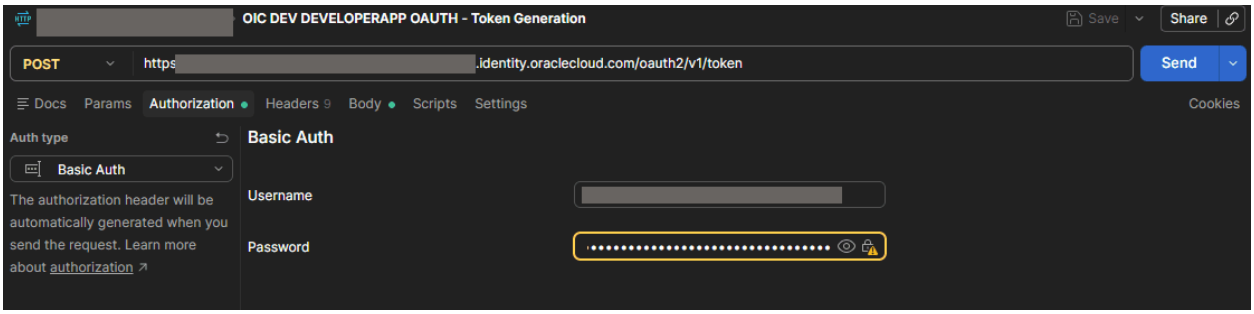
Before touching the connection, I ran four tests in Postman. A token only proves the scope; only a call into OIC proves the role.

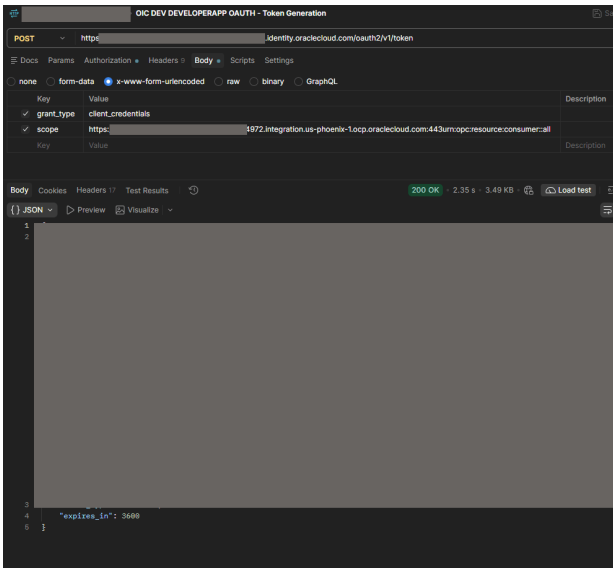
#	Test	Result	What it proves
1	Token for the consumer::all scope (client ID and vaulted secret sent as Basic Auth)	200, Bearer, 3600 s	The application, secret and endpoint scope are valid
2	Token for the /ic/api/ scope	200, Bearer, 3600 s	The application holds the Developer API scope
3	GET /ic/api/integration/v1/integrations?limit=5 on the design-time URL with token 2	200, integration list	OIC accepts the /ic/api/ token for reads
4	POST to an app-driven demo integration on the runtime URL with token 1	200, response echoes the client ID	OIC accepts the application for running integrations

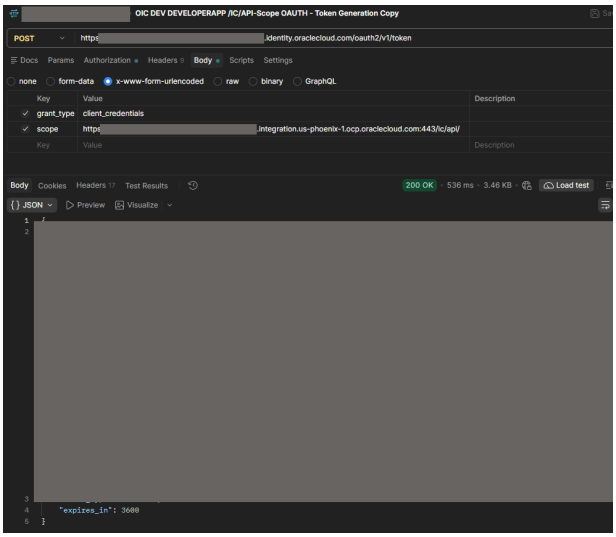
The token request uses the client ID and secret as Basic Auth, with grant_type=client_credentials and the scope in the body:

```
POST https://<identity-domain>.identity.oraclecloud.com/oauth2/v1/token
Authorization: Basic <client_id:client_secret>
Content-Type: application/x-www-form-urlencoded

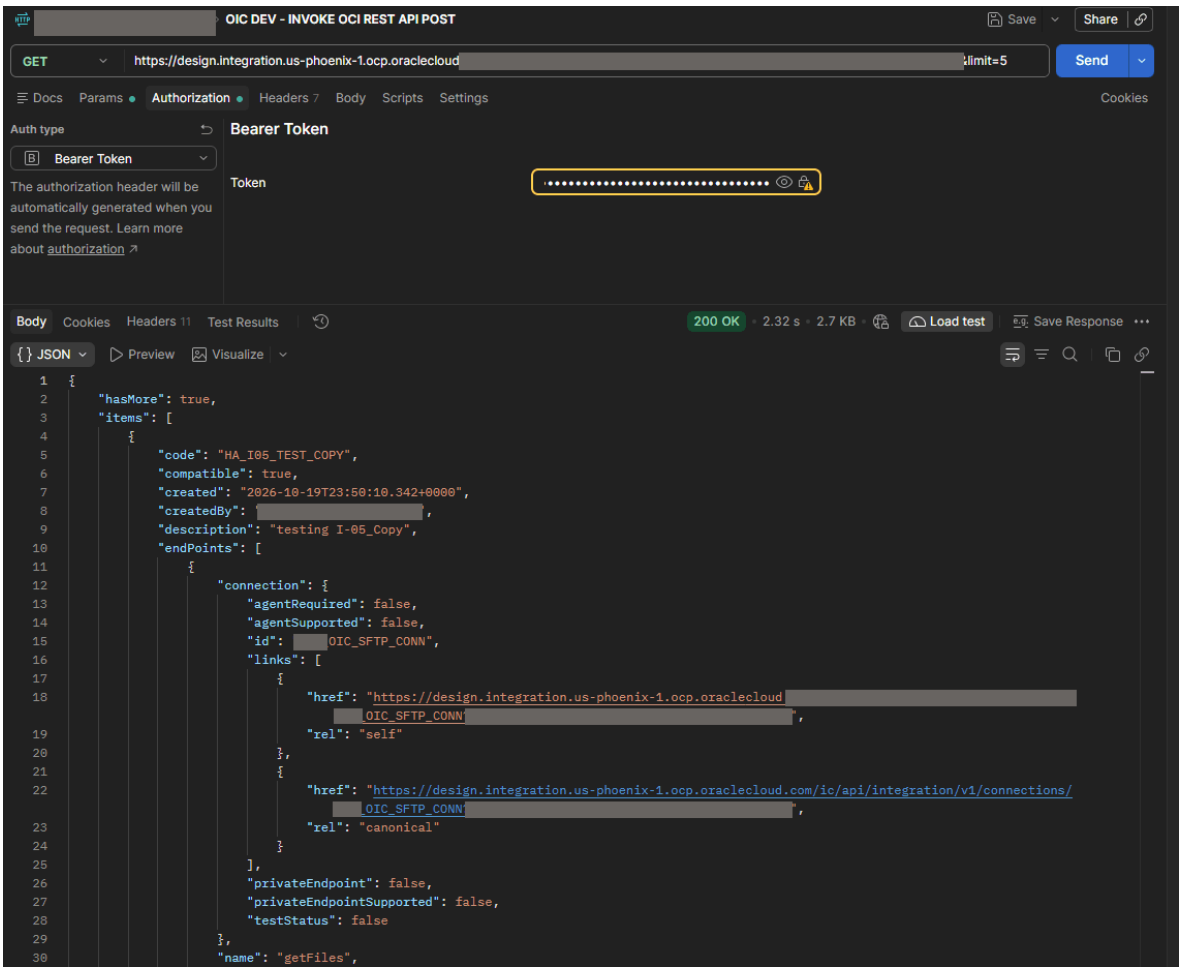
grant_type=client_credentials&scope=<OIC audience>urn:opc:resource:consumer::all
```



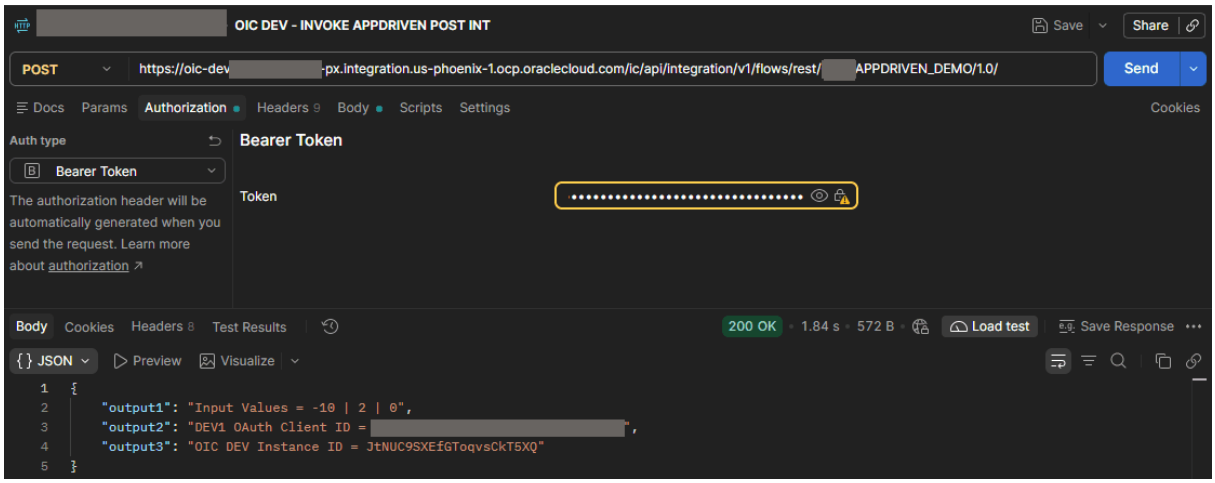




With the /ic/api/ token, the Developer API read returned the integration list.

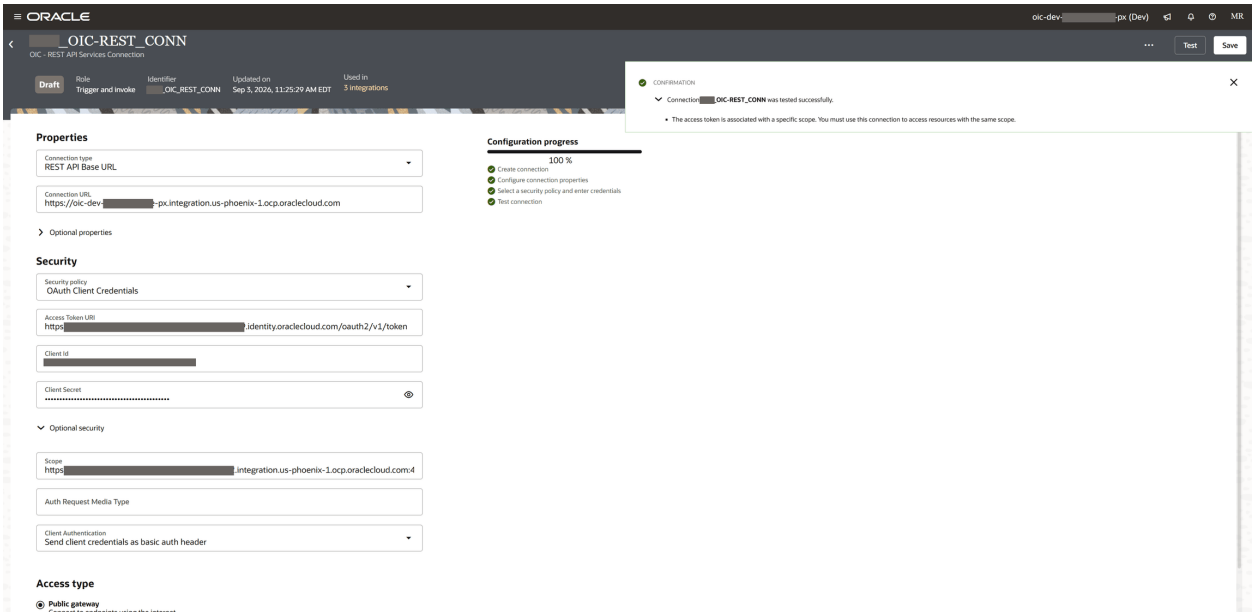


With the consumer::all token, the demo integration ran and echoed the application's client ID back in its response.

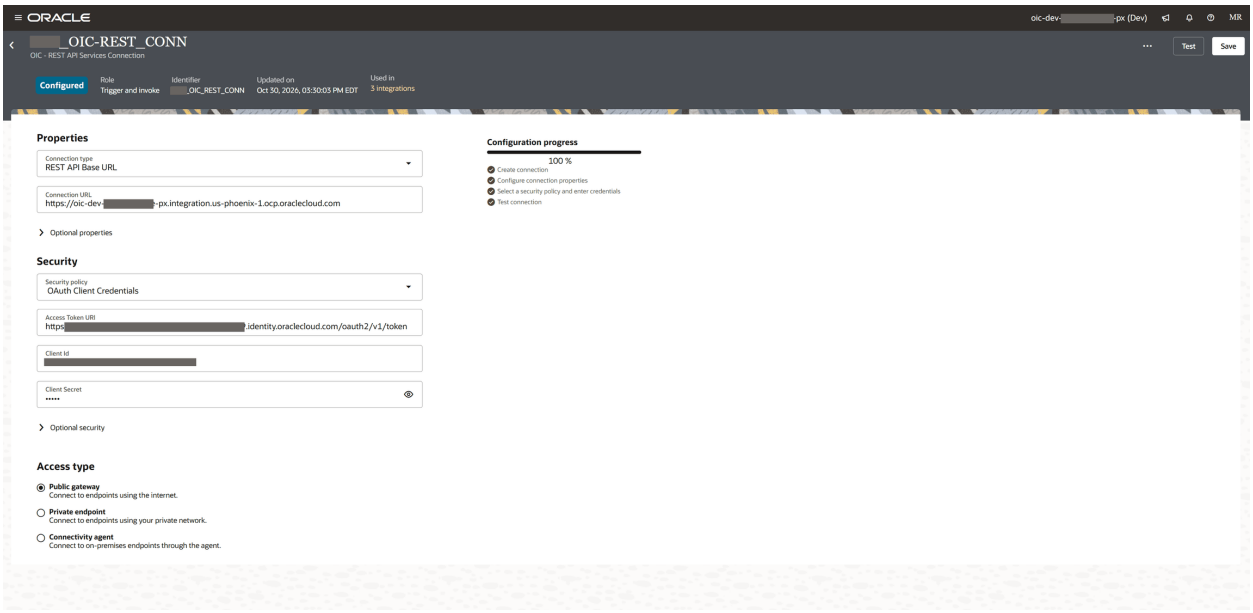


Part 5: Switch the connection and confirm

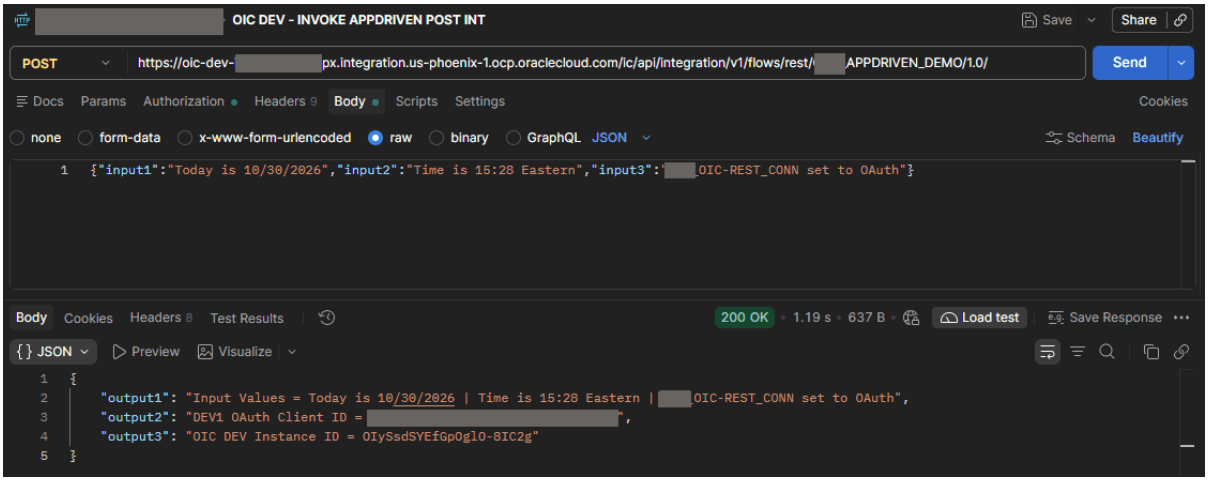
I changed the security policy on OIC-REST_CONN to OAuth Client Credentials, entered the token URL, client ID, vaulted secret and the consumer::all scope, and set Client Authentication to send the client credentials as a basic auth header. Test passed: the connection fetched a token for its scope exactly as an outbound call does.

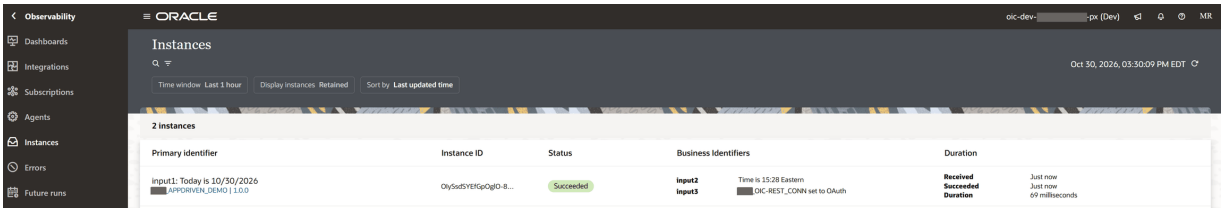


I saved the connection, and it showed Configured on OAuth Client Credentials with the same runtime URL.



Finally, I called the demo integration again after the switch and checked the instance in OIC monitoring: Succeeded.





What I learned

- Give each caller its own confidential application. OIC calling itself gets an internal developer application, and each external system gets its own invoke-only one, so a secret leak or change is contained and the caller is always traceable.
- Scopes and roles are checked separately. The token request succeeds as soon as the scope is granted, but the call into OIC fails without the right application role.

- A connection has one base URL and one scope, so OIC's runtime endpoints and its Developer APIs need separate connections, even with the same application.
- Never capture a secret on screen. If a secret value ever appears in a screenshot, regenerate it and create a new vault secret version before using it.

Configuration Summary

Setting	**Before**	**After (October 30, 2026)**
Security policy	Basic Authentication	OAuth Client Credentials
Credentials	Named user account and password	Client ID and the vaulted client secret
Access token URI	Not used	Identity domain token URL (/oauth2/v1/token)
Scope	Not used	<OIC audience>urn:opc:resource:consumer::all
Client authentication	Not used	Send client credentials as basic auth header
OIC role	The user's own roles	ServiceDeveloper, assigned to the application
Connection URL	Runtime host	Unchanged
Status	Configured	Test passed, saved, Configured